

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерных технологий

Лабораторная работа 4

«Аппроксимация функции методом наименьших квадратов»

по дисциплине «Вычислительная математика»

Вариант 13

Выполнила:

Павличенко Софья Алексеевна, Р3215

Проверила:

Малышева Татьяна Алексеевна

Санкт-Петербург 2025г.

Оглавление

Цель	3
Часть 1: Вычислительная реализация задачи.....	3
Линейное приближение	3
Часть 2: Программная реализация задачи.	5
Листинг программы	5
Результат работы программы.....	11
Выводы.....	13

Цель

Найти функцию, являющуюся наилучшим приближением заданной табличной функции по методу наименьших квадратов.

Часть 1: Вычислительная реализация задачи

$$f(x) = \frac{31x}{x^4 + 13}, \quad x \in [0,4], \quad h = 0.4$$

i	1	2	3	4	5	6	7	8	9	10	11
x_i	0	0.4	0.8	1.2	1.6	2.0	2.4	2.8	3.2	3.6	4.0
y_i	0	0.952	1.849	2.468	2.537	2.138	1.611	1.166	0.842	0.617	0.461

Линейное приближение

$$\varphi(x) = ax + b$$

$$SX = \sum_{i=1}^n x_i = 22, \quad SXX = \sum_{i=1}^n x_i^2 = 61.6, \quad SY = \sum_{i=1}^n y_i = 14.641, \quad SXY = \sum_{i=1}^n x_i y_i = 27.044$$

$$\begin{cases} aSXX + bSX = SXY \\ aSX + bn = SY \end{cases} \leftrightarrow \begin{cases} 61.6a + 22b = 27.044 \\ 22a + 11b = 14.64 \end{cases}$$

$$\Delta = SXX * n - SX * SX = 61.6 * 11 - 22 * 22 = 193.6$$

$$\Delta_1 = SXY * n - SX * SY = 27.044 * 11 - 22 * 14.64 = -24.596$$

$$\Delta_2 = SXX * SY - SX * SXY = 61.6 * 14.64 - 22 * 27.044 = 306.856$$

$$a = \frac{\Delta_1}{\Delta} = -0.127, \quad b = \frac{\Delta_2}{\Delta} = 1.585$$

$$P_1(x) = -0.127x + 1.585$$

i	1	2	3	4	5	6	7	8	9	10	11
x_i	0	0.4	0.8	1.2	1.6	2.0	2.4	2.8	3.2	3.6	4.0
y_i	0	0.952	1.849	2.468	2.537	2.138	1.611	1.166	0.842	0.617	0.461
$P_1(x) = -0.127x + 1.585$	1.585	1.534	1.483	1.433	1.382	1.331	1.280	1.229	1.179	1.128	1.077
ε_i	0	0.552	1.049	1.268	0.937	0.138	-0.789	-1.634	-2.358	-2.983	-3.539

$$\delta_{\text{лин}} = \sqrt{\frac{\sum_{i=1}^n (P_1(x_i) - y_i)^2}{n}} = \sqrt{\frac{\sum_{i=1}^n \varepsilon_i^2}{n}} = \sqrt{\frac{34.185}{11}} = 1.763$$

Квадратичное приближение

$$\varphi(x) = a_0 + a_1x + a_2x^2$$

$$SX = \sum_{i=1}^n x_i = 22, \quad SXX = \sum_{i=1}^n x_i^2 = 61.6, \quad SXXX = \sum_{i=1}^n x_i^3 = 193.6,$$

$$SXXXX = \sum_{i=1}^n x_i^4 = 648.525, \quad SY = \sum_{i=1}^n y_i = 14.641, \quad SXY = \sum_{i=1}^n x_i y_i = 27.044,$$

$$SXXY = \sum_{i=1}^n x_i^2 y_i = 62.352$$

$$\begin{cases} a_0 n + a_1 SX + a_2 SXX = SY \\ a_0 SX + a_1 SXX + a_2 SXXX = SXY \\ a_0 SXX + a_1 SXXX + a_2 SXXXX = SXXY \end{cases} \leftrightarrow \begin{cases} 11a_0 + 22a_1 + 61.6a_2 = 14.641 \\ 22a_0 + 61.6a_1 + 193.6a_2 = 27.044 \\ 61.6a_0 + 193.6a_1 + 648.525a_2 = 62.352 \end{cases}$$

$$\leftrightarrow \begin{cases} a_0 = \frac{14.641 - 22a_1 + 61.6a_2}{11} \\ 22a_0 + 61.6a_1 + 193.6a_2 = 27.044 \\ 61.6a_0 + 193.6a_1 + 648.525a_2 = 62.352 \end{cases}$$

$$\leftrightarrow \begin{cases} a_0 = \frac{14.641 - 22a_1 + 61.6a_2}{11} \\ a_1 = -18a_2 - 0.127 \\ 61.6a_0 + 193.6a_1 + 648.525a_2 = 62.352 \end{cases}$$

$$\leftrightarrow \begin{cases} a_0 = \frac{14.641 - 22a_1 + 61.6a_2}{11} \\ a_1 = -18a_2 - 0.127 \\ a_1 = -14.112a_2 - 0.279 \end{cases} \leftrightarrow \begin{cases} a_0 = \frac{14.641 - 22a_1 + 61.6a_2}{11} \\ a_1 = -18a_2 - 0.127 \\ -18a_2 - 0.127 = -14.112a_2 - 0.279 \end{cases}$$

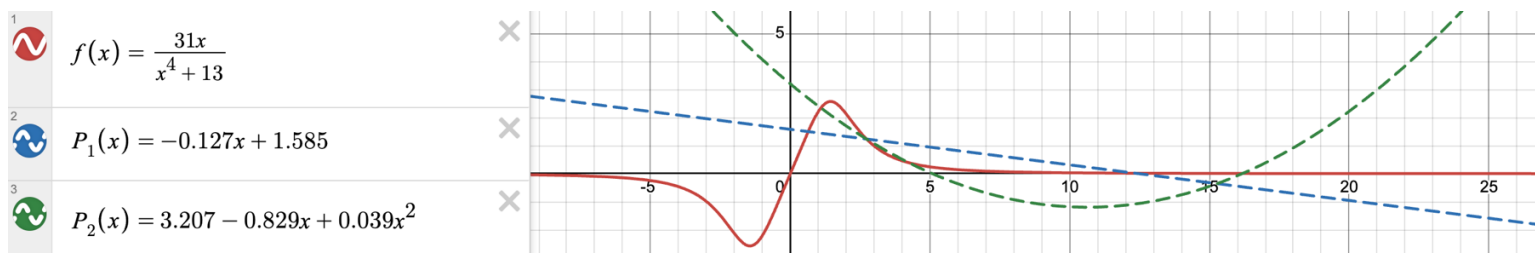
$$\leftrightarrow \begin{cases} a_0 = \frac{14.641 - 22a_1 + 61.6a_2}{11} \\ a_1 = -18a_2 - 0.127 \\ a_2 = 0.039 \end{cases} \leftrightarrow \begin{cases} a_0 = 3.207 \\ a_1 = -0.829 \\ a_2 = 0.039 \end{cases}$$

$$P_2(x) = 3.207 - 0.829x + 0.039x^2$$

i	1	2	3	4	5	6	7	8	9	10	11
x_i	0	0.4	0.8	1.2	1.6	2.0	2.4	2.8	3.2	3.6	4.0
y_i	0	0.952	1.849	2.468	2.537	2.138	1.611	1.166	0.842	0.617	0.461
$P_2(x)$	3.207	2.882	2.569	2.268	1.980	1.705	1.442	1.192	0.954	0.728	0.515
ε_i	3.207	1.930	0.720	-0.200	-0.557	-0.433	-0.169	0.026	0.112	0.111	0.054

$$\delta_{\text{квадр}} = \sqrt{\frac{\sum_{i=1}^n (P_2(x_i) - y_i)^2}{n}} = \sqrt{\frac{\sum_{i=1}^n \varepsilon_i^2}{n}} = \sqrt{\frac{15.12}{11}} = 1.172$$

$\delta_{\text{квадр}} < \delta_{\text{лин}} \Rightarrow$ **квадратичное** приближение лучше.



Часть 2: Программная реализация задачи

Листинг программы

```
import matplotlib.pyplot as plt
import numpy as np

def plot_all_approximations():
    x_vals = [x for x, y in table]
    y_vals = [y for x, y in table]

    a = min(x_vals)
    b = max(x_vals)
    margin_x = 0.1 * (b - a)
    if a > 0:
        while a - margin_x < 0:
            margin_x /= 2
    margin_y = 0.1 * (max(y_vals) - min(y_vals))
    x_range = np.linspace(a - margin_x, b + margin_x, 1000)

    for name, func, _ in approximations:
        y_range = [func(x) for x in x_range]
        plt.plot(x_range, y_range, label=name)

    plt.scatter(x_vals, y_vals, label="Исходные точки")

    plt.xlabel("x")
    plt.ylabel("y")
    plt.title("Графики аппроксимаций")
    plt.ylim(min(y_vals) - margin_y, max(y_vals) + margin_y)
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.show()

def print_approximation_result(name, func_view, coeffs, S, R2, std_dev,
    phi_values, eps_values, func):
    print('=====')
    print(f'{name.upper()} АППРОКСИМАЦИЯ: {func_view}')
    print('=====')
    print('Коэффициенты:')
```

```

coef_names = ['a', 'b', 'c', 'd']
for i, coef in enumerate(coeffs):
    label = coef_names[i] if i < len(coef_names) else f'k{i}'
    print(f' {label} = {coef:.6f}')
print()

print(f"{'x_i':>10} {'y_i':>10} {'φ(x_i)':>15} {'ε_i':>15}")
for i, (x, y) in enumerate(table):
    print(f"{x:10.4f} {y:10.4f} {phi_values[i]:15.6f} {eps_values[i]:15.6f}")

print(f'\nМера отклонения: S = {S:.6f}')
print(f'Среднеквадратичное отклонение: δ = {std_dev:.6f}')
print(f'Коэффициент детерминации: R² = {R2:.6f}')

if R2 >= 0.95:
    interpretation = "высокая точность аппроксимации"
elif R2 >= 0.75:
    interpretation = "удовлетворительная аппроксимация"
elif R2 >= 0.5:
    interpretation = "слабая аппроксимация"
else:
    interpretation = "недостаточная точность аппроксимации"

print(f'Интерпретация R²: {interpretation}')

approximations.append((name, func, std_dev))

def calculate_metrics(func):
    phi_values = [func(x) for x, y in table]
    average_phi = sum(phi_values) / n
    eps_values = [func(x) - y for x, y in table]

    S = sum(eps**2 for eps in eps_values)
    standard_deviation = (S / n) ** 0.5
    R2 = 1 - S / sum((y - average_phi) ** 2 for x, y in table)

    return func, S, R2, standard_deviation, phi_values, eps_values

def linear_regression(x_values, y_values):
    sx = sum(x_values)
    sxx = sum(x**2 for x in x_values)
    sy = sum(y_values)
    sxy = sum(x * y for x, y in zip(x_values, y_values))

    delta = sxx * n - sx * sx
    delta1 = sxy * n - sx * sy
    delta2 = sxx * sy - sx * sxy

    a = delta1 / delta
    b = delta2 / delta
    return a, b

def linear_approximation():
    average_x = sum(list(x for x, y in table)) / n
    average_y = sum(list(y for x, y in table)) / n

    r = (sum((x - average_x) * (y - average_y) for x, y in table) /
         (sum((x - average_x)**2 for x, y in table) * sum((y - average_y)**2 for

```

```

x, y in table))**0.5)

a, b = linear_regression([x for x, y in table], [y for x, y in table])
P1 = lambda x: a*x + b

return (a, b), *calculate_metrics(P1), r

def quadratic_approximation():
    sx = sum(list(x for x, y in table))
    sxx = sum(list(x**2 for x, y in table))
    sxxx = sum(list(x**3 for x, y in table))
    sxxxx = sum(list(x**4 for x, y in table))
    sy = sum(list(y for x, y in table))
    sxy = sum(list(x * y for x, y in table))
    sxxxy = sum(list(x**2 * y for x, y in table))

    A = [[sxxxx, sxxx, sxx],
          [sxxx, sxx, sx],
          [sxx, sx, n]]
    B = [sxxxy, sxy, sy]

    a, b, c = np.linalg.solve(np.array(A, dtype=float), np.array(B, dtype=float))
    P2 = lambda x: c + b*x + a*x**2

    return (a, b, c), *calculate_metrics(P2)

def cubic_approximation():
    sx = sum(list(x for x, y in table))
    sxx = sum(list(x**2 for x, y in table))
    sxxx = sum(list(x**3 for x, y in table))
    sxxxx = sum(list(x**4 for x, y in table))
    sxxxxx = sum(list(x**5 for x, y in table))
    sxxxxxx = sum(list(x**6 for x, y in table))
    sy = sum(list(y for x, y in table))
    sxy = sum(list(x * y for x, y in table))
    sxxxy = sum(list(x**2 * y for x, y in table))
    sxxxxy = sum(list(x**3 * y for x, y in table))

    A = [[sxxxxxx, sxxxxx, sxxxx, sxxx],
          [sxxxxx, sxxxx, sxxx, sxx],
          [sxxxx, sxxx, sxx, sx],
          [sxxx, sxx, sx, n]]
    B = [sxxxxy, sxxxy, sxy, sy]

    a, b, c, d = np.linalg.solve(np.array(A, dtype=float), np.array(B,
dtype=float))
    P3 = lambda x: a*x**3 + b*x**2 + c*x + d

    return (a, b, c, d), *calculate_metrics(P3)

def power_approximation():
    X = [np.log(x) for x, y in table]
    Y = [np.log(y) for x, y in table]
    A, B = linear_regression(X, Y)

    a = np.exp(B)
    b = A
    P4 = lambda x: a * x**b

```

```

    return (a, b), *calculate_metrics(P4)

def exponential_approximation():
    X = [x for x, y in table]
    Y = [np.log(y) for x, y in table]
    A, B = linear_regression(X, Y)

    a = np.exp(B)
    b = A
    P5 = lambda x: a * np.exp(b*x)

    return (a, b), *calculate_metrics(P5)

def logarithmic_approximation():
    X = [np.log(x) for x, y in table]
    Y = [y for x, y in table]
    a, b = linear_regression(X, Y)

    P6 = lambda x: a*np.log(x) + b

    return (a, b), *calculate_metrics(P6)

# --- Ввод данных ---

n = 0
table = []
approximations = []

print('Вычислительная математика. Лабораторная работа 4: "Аппроксимация функции методом наименьших квадратов".\n')

print("Как вы хотите ввести исходные данные?\n1 - с консоли\n2 - с файла")
v = input()
while v not in {'1', '2'}:
    print('Выберите первый (1) или второй (2) вариант')
    v = input()

print()
if v == '1':
    n = int(input("Сколько точек вы хотите ввести (от 8 до 12)? "))
    while not n in range(8, 13):
        n = int(input("Выберите значение от 8 до 12: "))
    print("Введите построчно значения x и y через пробел:")
    for i in range(n):
        table.append(tuple(map(float, input().split())))
else:
    print("Введите название файла:")
    with open(input().strip(), 'r') as file:
        lines = file.readlines()
        n = int(lines[0])
        for i in range(n):
            table.append(tuple(map(float, lines[i + 1].split()))))
print()

# --- Линейное приближение ---

```



```

(a, b), func, S, R2, std_dev, phi_values, eps_values, r = linear_approximation()

print_approximation_result(
    "Линейная",
    "y = ax + b",
    (a, b),
    S, R2, std_dev,
    phi_values, eps_values,
    func
)
print(f'Коэффициент корреляции Пирсона: r = {r:.6f}')

if abs(r) == 1:
    interpretation = "строгая линейная функциональная зависимость"
elif r >= 0.9:
    interpretation = "связь весьма высокая"
elif r >= 0.7:
    interpretation = "связь высокая"
elif r >= 0.5:
    interpretation = "связь заметная"
elif r >= 0.3:
    interpretation = "связь умеренная"
elif r > 0:
    interpretation = "связь слабая"
else:
    interpretation = "связь отсутствует"

print(f'Интерпретация коэффициента корреляции Пирсона: {interpretation}\n\n')

# --- Квадратичное приближение ---

(a, b, c), func, S, R2, std_dev, phi_values, eps_values =
quadratic_approximation()
print_approximation_result(
    "Квадратичная",
    "y = ax^2 + bx + c",
    (a, b, c),
    S, R2, std_dev,
    phi_values, eps_values,
    func
)
print('\n\n')

# --- Кубическое приближение ---

(a, b, c, d), func, S, R2, std_dev, phi_values, eps_values =
cubic_approximation()
print_approximation_result(
    "Кубическая",
    "y = ax^3 + bx^2 + cx + d",
    (a, b, c, d),
    S, R2, std_dev,
    phi_values, eps_values,
    func
)
print('\n\n')

# --- Степенное приближение ---

```

```

if any(x <= 0 or y <= 0 for x, y in table):
    print("Для степенной аппроксимации все x и y должны быть > 0\n\n")
else:
    (a, b), func, S, R2, std_dev, phi_values, eps_values = power_approximation()
    print_approximation_result(
        "Степенная",
        "y = a * x^b",
        (a, b),
        S, R2, std_dev,
        phi_values, eps_values,
        func
    )
    print('\n\n')

# --- Экспоненциальное приближение ---

if any(y <= 0 for x, y in table):
    print("Для экспоненциальной аппроксимации все y должны быть > 0\n\n")
else:
    (a, b), func, S, R2, std_dev, phi_values, eps_values =
exponential_approximation()
    print_approximation_result(
        "Экспоненциальная",
        "y = a * e^(bx)",
        (a, b),
        S, R2, std_dev,
        phi_values, eps_values,
        func
    )
    print('\n\n')

# --- Логарифмическое приближение ---

if any(x <= 0 for x, y in table):
    print("Для логарифмической аппроксимации все x должны быть > 0\n\n")
else:
    (a, b), func, S, R2, std_dev, phi_values, eps_values =
logarithmic_approximation()
    print_approximation_result(
        "Логарифмическая",
        "y = a * ln(x) + b",
        (a, b),
        S, R2, std_dev,
        phi_values, eps_values,
        func
    )
    print('\n\n')

plot_all_approximations()

# --- Лучшие приближения ---

min_std_dev = min(std for _, _, std in approximations)
best_approximations = [(name, std) for name, _, std in approximations if abs(std
- min_std_dev) < 1e-8]

print('=====')

```

```

print('ЛУЧШАЯ(-ИЕ) АППРОКСИМАЦИЯ(-И):')
for name, std in best_approximations:
    print(f'- {name.upper()} (δ = {std:.6f})')
print('=====')

```

Результат работы программы

Вычислительная математика. Лабораторная работа 4: "Аппроксимация функции методом наименьших квадратов".

Как вы хотите ввести исходные данные?

1 - с консоли

2 - с файла

1

Сколько точек вы хотите ввести (от 8 до 12)? 10

Введите построчно значения x и y через пробел:

0.01 1.0

0.02 1.3

0.03 1.5

0.05 1.9

0.07 2.3

0.1 2.8

0.2 3.4

0.4 4.0

0.7 4.3

1.0 4.5

ЛИНЕЙНАЯ АППРОКСИМАЦИЯ: $y = ax + b$

Коэффициенты:

a = 3.298631

b = 1.848953

x _i	y _i	φ(x _i)	ε _i
0.0100	1.0000	1.881939	0.881939
0.0200	1.3000	1.914926	0.614926
0.0300	1.5000	1.947912	0.447912
0.0500	1.9000	2.013885	0.113885
0.0700	2.3000	2.079857	-0.220143

...

ЛОГАРИФИЧЕСКАЯ АППРОКСИМАЦИЯ: $y = a * \ln(x) + b$

Коэффициенты:

$a = 0.832889$

$b = 4.604294$

x_i	y_i	$\varphi(x_i)$	ε_i
0.0100	1.0000	0.768700	-0.231300
0.0200	1.3000	1.346015	0.046015
0.0300	1.5000	1.683722	0.183722
0.0500	1.9000	2.109183	0.209183
0.0700	2.3000	2.389427	0.089427
0.1000	2.8000	2.686497	-0.113503
0.2000	3.4000	3.263812	-0.136188
0.4000	4.0000	3.841126	-0.158874
0.7000	4.3000	4.307224	0.007224
1.0000	4.5000	4.604294	0.104294

Мера отклонения: $S = 0.208726$

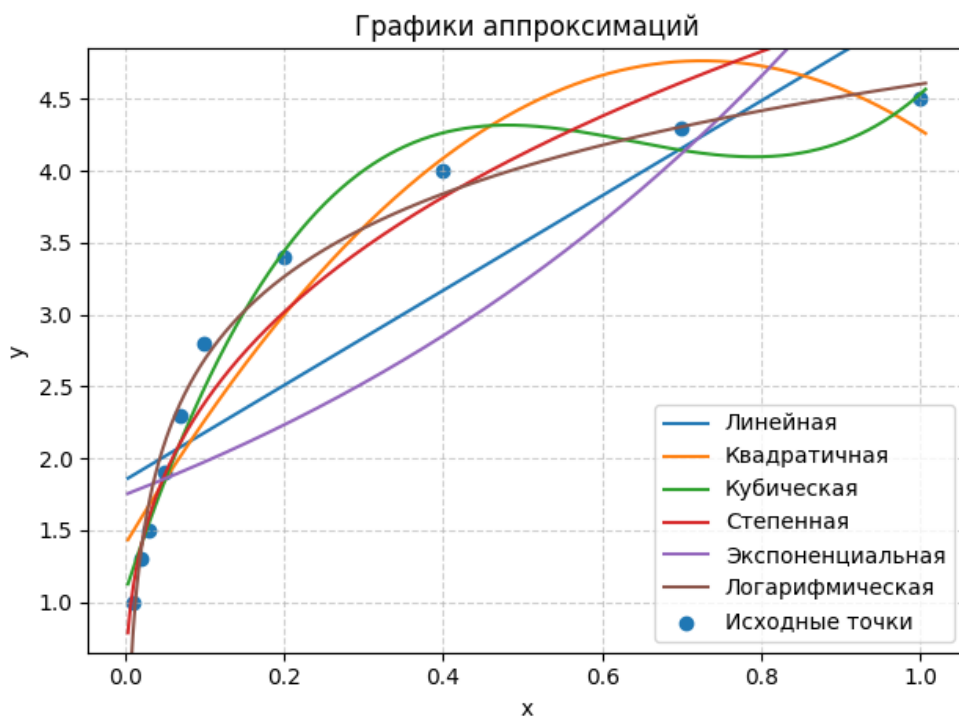
Среднеквадратичное отклонение: $\delta = 0.144473$

Коэффициент детерминации: $R^2 = 0.986159$

Интерпретация R^2 : высокая точность аппроксимации

ЛУЧШАЯ(-ИЕ) АППРОКСИМАЦИЯ(-И):

- ЛОГАРИФИЧЕСКАЯ ($\delta = 0.144473$)



Выводы

В ходе работы я изучила метод наименьших квадратов для аппроксимации функции. Вручную выполнила аппроксимацию с использованием линейного и квадратичного приближения. На Python программно выполнила аппроксимацию с использованием линейного, квадратичного, кубического, экспоненциального, логарифмического и степенного приближения и построила графики исходной функции и аппроксимаций. Исследование позволило определить наилучшее приближение, вычислить среднеквадратические отклонения и коэффициент корреляции Пирсона для линейной зависимости.